

Problemas Propuestos - MongoDB Consola

Resolución de los problemas 1.1 a 1.11 (guía: Crear Base de Datos por Consola)

Alumno: Sebastián

Todos los comandos se escriben en el shell de MongoDB (mongosh) con la base **base1** activa (use base1). Para cargar varios documentos de una vez se usa **insertMany**, que da el mismo resultado que repetir **insertOne** por cada documento. Debajo de algunas consultas se indica el resultado esperado.

1.1 Problemas propuestos 1.1 - Insertar documentos y campo `_id`

1. Insertar 2 documentos en la colección `clientes` con '`_id`' no repetidos.

```
use base1

db.clientes.insertOne({
  _id: 1,
  nombre: 'Lopez Marcos',
  domicilio: 'Colon 111',
  provincia: 'Cordoba'
})

db.clientes.insertOne({
  _id: 2,
  nombre: 'Perez Ana',
  domicilio: 'San Martin 222',
  provincia: 'Santa Fe'
})
```

Cada inserción devuelve { acknowledged: true, insertedId: 1 } (y luego insertedId: 2).

2. Intentar insertar otro documento con clave repetida.

```
// El _id: 1 ya existe, así que MongoDB rechaza la inserción
db.clientes.insertOne({
  _id: 1,
  nombre: 'Gomez Laura',
  domicilio: 'Belgrano 333',
  provincia: 'Mendoza'
})
```

Resultado: error E11000 duplicate key error (el campo errmsg informa la clave duplicada). El documento no se inserta.

3. Mostrar todos los documentos de la colección libros.

```
db.libros.find()
```

Muestra los documentos de la colección libros cargados en los ejemplos de la guía (cada uno con su campo `_id`).

1.2

Problemas propuestos 1.2 - Crear y borrar bases de datos y colecciones

1. Crear una base de datos llamada "blog".

```
use blog
```

Con `use` la base queda activa, pero MongoDB no la crea realmente hasta que se guarda el primer documento.

2. Agregar una colección llamada "posts" e insertar 1 documento con una estructura a su elección.

```
db.posts.insertOne({
  _id: 1,
  titulo: 'Mi primer post',
  autor: 'Sebastian',
  contenido: 'Hoy empiezo a aprender MongoDB desde la consola.',
  etiquetas: ['mongodb', 'nosql'],
  fecha: new Date()
})
```

3. Mostrar todas las bases de datos actuales.

```
show dbs
```

Ahora la base `blog` aparece en la lista junto con `admin`, `config`, `local` y las demás que existan.

4. Eliminar la colección "posts".

```
db.posts.drop()
```

Devuelve `true`. Se puede comprobar con `show collections`.

5. Eliminar la base de datos "blog" y mostrar las bases de datos existentes.

```
use blog
db.dropDatabase()
show dbs
```

Devuelve `{ ok: 1, dropped: 'blog' }` y `blog` ya no figura en la lista.

1.3

Problemas propuestos 1.3 - Consultas con find y operadores relacionales

1. Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente) y cargar 6 documentos.

```
use base1
db.articulos.drop()

db.articulos.insertMany([
  { _id: 1, nombre: 'MULTIFUNCION HP DESKJET 2675',
    rubro: 'impresora', precio: 3000, stock: 20 },
  { _id: 2, nombre: 'MULTIFUNCION EPSON EXPRESSION XP241',
    rubro: 'impresora', precio: 3700, stock: 5 },
  { _id: 3, nombre: 'LED 19 PHILIPS',
    rubro: 'monitor', precio: 4500, stock: 2 },
  { _id: 4, nombre: 'LED 22 PHILIPS',
    rubro: 'monitor', precio: 5700, stock: 4 },
  { _id: 5, nombre: 'LED 27 PHILIPS',
    rubro: 'monitor', precio: 12000, stock: 1 },
  { _id: 6, nombre: 'LOGITECH M90',
    rubro: 'mouse', precio: 300, stock: 4 }
])
```

Este mismo script se reutiliza en los problemas 1.4, 1.5, 1.6 y 1.8.

2. Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

3. Imprimir todos los documentos de la colección 'articulos' que no son impresoras.

```
db.articulos.find({ rubro: { $ne: 'impresora' } })
```

Devuelve los 3 monitores y el mouse (_id 3, 4, 5 y 6).

4. Imprimir todos los artículos que pertenecen al rubro de 'mouse'.

```
db.articulos.find({ rubro: 'mouse' })
```

Devuelve el LOGITECH M90 (_id 6).

5. Imprimir todos los artículos con un precio mayor o igual a 5000.

```
db.articulos.find({ precio: { $gte: 5000 } })
```

Devuelve LED 22 PHILIPS (5700) y LED 27 PHILIPS (12000).

6. Imprimir todas las impresoras que tienen un precio mayor o igual a 3500.

```
db.articulos.find({ rubro: 'impresora', precio: { $gte: 3500 } })
```

Devuelve solo la MULTIFUNCION EPSON EXPRESSION XP241 (3700).

7. Imprimir todos los artículos cuyo stock se encuentra comprendido entre 0 y 4.

```
db.articulos.find({ stock: { $gte: 0, $lte: 4 } })
```

Devuelve LED 19 (stock 2), LED 22 (4), LED 27 (1) y LOGITECH M90 (4).

1.4 Problemas propuestos 1.4 - Borrar con deleteOne y deleteMany

1. Crear la colección 'articulos' (eliminar la colección previamente) y cargar los 6 documentos del problema 1.3.

```
// Se ejecuta el mismo script del problema 1.3
use base1
db.articulos.drop()
// ... db.articulos.insertMany([ ...los 6 documentos... ])
```

Antes de resolver este problema se vuelve a cargar la colección **articulos** con el mismo script del problema 1.3 (punto 1).

2. Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

3. Borrar los documentos de la colección 'articulos' cuyo rubro son impresoras, utilizar las dos sintaxis que permite MongoDB.

```
// Sintaxis 1: igualdad directa
db.articulos.deleteMany({ rubro: 'impresora' })

// Sintaxis 2: con el operador $eq
db.articulos.deleteMany({ rubro: { $eq: 'impresora' } })
```

La primera elimina las 2 impresoras (deletedCount: 2). Como las dos sintaxis son equivalentes, para probar la segunda conviene volver a cargar la colección; si se ejecuta justo después de la primera, devuelve deletedCount: 0.

4. Borrar todos los artículos que tienen un _id mayor o igual a 5.

```
db.articulos.deleteMany({ _id: { $gte: 5 } })
```

Elimina LED 27 PHILIPS (_id 5) y LOGITECH M90 (_id 6).

1.5

Problemas propuestos 1.5 - Modificar un documento con updateOne

1. Crear la colección 'articulos' (eliminar la colección previamente) y cargar los 6 documentos del problema 1.3.

```
// Se ejecuta el mismo script del problema 1.3
use base1
db.articulos.drop()
// ... db.articulos.insertMany([ ...los 6 documentos... ])
```

Antes de resolver este problema se vuelve a cargar la colección **articulos** con el mismo script del problema 1.3 (punto 1).

2. Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

3. Modificar el precio del mouse 'LOGITECH M90'.

```
// El precio nuevo (350) es un ejemplo, se puede poner cualquier valor
db.articulos.updateOne(
  { nombre: 'LOGITECH M90' },
  { $set: { precio: 350 } }
)
```

Devuelve matchedCount: 1, modifiedCount: 1. El precio pasa de 300 a 350.

4. Fijar el stock en 0 del artículo cuyo _id es 6.

```
db.articulos.updateOne({ _id: 6 }, { $set: { stock: 0 } })
```

5. Agregar el campo proveedores con el array ['Martinez','Gutierrez'] para el artículo cuyo _id es 6.

```
db.articulos.updateOne(
  { _id: 6 },
  { $set: { proveedores: ['Martinez', 'Gutierrez'] } }
)
```

Como el campo no existía, \$set lo crea dentro del documento.

6. Eliminar el campo proveedores para el artículo cuyo _id es 6.

```
db.articulos.updateOne({ _id: 6 }, { $unset: { proveedores: '' } })

// Para comprobar el resultado
db.articulos.find({ _id: 6 })
```

\$unset elimina el campo por completo (no lo deja vacío).

1.6

Problemas propuestos 1.6 - Modificar varios documentos con updateMany

1. Crear la colección 'articulos' (eliminar la colección previamente) y cargar los 6 documentos del problema 1.3.

```
// Se ejecuta el mismo script del problema 1.3
use base1
db.articulos.drop()
// ... db.articulos.insertMany([ ...los 6 documentos... ])
```

Antes de resolver este problema se vuelve a cargar la colección **articulos** con el mismo script del problema 1.3 (punto 1).

2. Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

3. Fijar el stock en cero para todos los artículos del rubro monitor.

```
db.articulos.updateMany({ rubro: 'monitor' }, { $set: { stock: 0 } })
```

Modifica los 3 monitores (matchedCount: 3, modifiedCount: 3).

4. Agregar un campo llamado 'pedir' con el valor true para todos los artículos que tienen el campo stock en 0.

```
db.articulos.updateMany({ stock: 0 }, { $set: { pedir: true } })
```

Quedan marcados los 3 monitores, que son los únicos con stock en 0.

5. Eliminar el campo 'pedir' de todos los documentos.

```
// El filtro vacío {} alcanza a todos los documentos de la colección
db.articulos.updateMany({}, { $unset: { pedir: '' } })
```

1.7

Problemas propuestos 1.7 - Operadores lógicos \$and, \$or y \$not

1. Crear la colección 'medicamentos' en la base de datos 'base1' (eliminar la colección previamente) y cargar 6 documentos.

```
use base1
db.medicamentos.drop()

db.medicamentos.insertMany([
  { _id: 1, nombre: 'Sertal', laboratorio: 'Roche', precio: 5.2, cantidad: 100 },
  { _id: 2, nombre: 'Buscapina', laboratorio: 'Roche', precio: 4.10, cantidad: 200 },
  { _id: 3, nombre: 'Amoxidal 500', laboratorio: 'Bayer', precio: 15.60, cantidad: 100 },
  { _id: 4, nombre: 'Paracetamol 500', laboratorio: 'Bago', precio: 1.90, cantidad: 200 },
  { _id: 5, nombre: 'Bayaspirina', laboratorio: 'Bayer', precio: 2.10, cantidad: 150 },
  { _id: 6, nombre: 'Amoxidal jarabe', laboratorio: 'Bayer', precio: 5.10, cantidad: 50 }
])
```

2. Imprimir todos los documentos de la colección 'medicamentos'.

```
db.medicamentos.find()
```

3. Recupere los medicamentos cuyo laboratorio sea 'Roche' y cuyo precio sea menor a 5.

```
db.medicamentos.find({ laboratorio: 'Roche', precio: { $lt: 5 } })

// Es lo mismo que escribir el $and de forma explícita
db.medicamentos.find({ $and: [ { laboratorio: 'Roche' }, { precio: { $lt: 5 } } ] })
```

Devuelve solo Buscapina (4.10). Sertal es de Roche pero cuesta 5.2.

4. Recupere los medicamentos cuyo laboratorio sea 'Roche' o cuyo precio sea menor a 5.

```
db.medicamentos.find({
  $or: [ { laboratorio: 'Roche' }, { precio: { $lt: 5 } } ]
})
```

Devuelve Sertal, Buscapina, Paracetamol 500 y Bayaspirina.

5. Muestre todos los medicamentos cuyo laboratorio NO sea "Bayer".

```
db.medicamentos.find({ laboratorio: { $ne: 'Bayer' } })

// Otra forma, usando $not
db.medicamentos.find({ laboratorio: { $not: { $eq: 'Bayer' } } })
```

Devuelve Sertal, Buscapina y Paracetamol 500.

6. Muestre todos los medicamentos cuyo laboratorio sea "Bayer" y cuya cantidad NO sea = 100.

```
db.medicamentos.find({ laboratorio: 'Bayer', cantidad: { $ne: 100 } })
```

Devuelve Bayaspirina (150) y Amoxidal jarabe (50). Amoxidal 500 queda afuera porque tiene cantidad 100.

7. Elimine todos los documentos de la colección medicamentos cuyo laboratorio sea igual a "Bayer" y su precio sea mayor a 10.

```
db.medicamentos.deleteMany({ laboratorio: 'Bayer', precio: { $gt: 10 } })
```

Elimina solo Amoxidal 500 (15.60): deletedCount: 1.

8. Cambie la cantidad por 200, a todos los medicamentos de "Roche" cuyo precio sea mayor a 5.

```
db.medicamentos.updateMany(
  { laboratorio: 'Roche', precio: { $gt: 5 } },
  { $set: { cantidad: 200 } }
)
```

Modifica solo Sertal (5.2), cuya cantidad pasa de 100 a 200.

9. Borre los medicamentos cuyo laboratorio sea "Bayer" o cuyo precio sea menor a 3.

```
db.medicamentos.deleteMany({
  $or: [ { laboratorio: 'Bayer' }, { precio: { $lt: 3 } } ]
})
```

Siguiendo el orden de los puntos anteriores elimina Bayaspirina, Amoxidal jarabe y Paracetamol 500 (deletedCount: 3). Quedan Sertal y Buscapina.

1.8 Problemas propuestos 1.8 - Cursores y projection en find

1. Crear la colección 'articulos' (eliminar la colección previamente) y cargar los 6 documentos.

```
// Se ejecuta el mismo script del problema 1.3
use base1
db.articulos.drop()
// ... db.articulos.insertMany([ ...los 6 documentos... ])
```

Antes de resolver este problema se vuelve a cargar la colección **articulos** con el mismo script del problema 1.3 (punto 1).

2. Imprimir todos los documentos de la colección 'articulos', mostrar solo los campos _id y nombre.

```
// El _id se muestra por defecto, solo se pide el campo nombre
db.articulos.find({}, { nombre: 1 })
```

3. Imprimir todos los documentos de la colección 'articulos' que son impresoras, mostrar solo los campos nombre y precio.

```
// _id: 0 oculta el _id para que salgan únicamente nombre y precio
db.articulos.find({ rubro: 'impresora' }, { _id: 0, nombre: 1, precio: 1 })
```

4. Imprimir todas las impresoras que tienen un precio mayor o igual a 3500. Solo mostrar los campos _id, nombre, precio y stock.

```
db.articulos.find(
  { rubro: 'impresora', precio: { $gte: 3500 } },
  { nombre: 1, precio: 1, stock: 1 }
)
```

Devuelve solo la EPSON EXPRESSION XP241 con los campos _id, nombre, precio y stock.

5. Imprimir todos los documentos de la colección 'articulos' que son monitor, mostrar solo los campos nombre y precio ordenados de menor a mayor.

```
db.articulos.find({ rubro: 'monitor' }, { _id: 0, nombre: 1, precio: 1 }).sort({ precio: 1 })
```

Orden de salida: LED 19 (4500), LED 22 (5700) y LED 27 (12000). Con sort({ precio: 1 }) el orden es ascendente.

1.9

Problemas propuestos 1.9 - Documentos embebidos (campos de tipo documento)

1. Crear la colección 'libros' en la base de datos 'base1' (eliminar la colección previamente) y cargar 4 documentos.

```
use base1
db.libros.drop()

db.libros.insertMany([
  { _id: 1, titulo: 'El aleph',
    autor: { nombre: 'Borges', nacionalidad: 'Argentina' },
    editorial: ['Siglo XXI', 'Planeta'], precio: 20, cantidad: 50 },
  { _id: 2, titulo: 'Martin Fierro',
    autor: { nombre: 'Jose Hernandez', nacionalidad: 'Argentina' },
    editorial: ['Siglo XXI'], precio: 50, cantidad: 12 },
  { _id: 3, titulo: 'Aprenda PHP',
    autor: { nombre: 'Mario Molina', nacionalidad: 'Española' },
    editorial: ['Siglo XXI', 'Planeta'], precio: 50, cantidad: 20 },
  { _id: 4, titulo: 'Java en 10 minutos',
    autor: { nombre: 'Sergio Barros', nacionalidad: 'Española' },
    editorial: ['Siglo XXI'], precio: 45, cantidad: 1 }
])
```

En el enunciado del PDF el cuarto documento no tiene el campo titulo y pone 'Java en 10 minutos' como nombre del autor (parece un error de tipeo). Acá se corrigió: titulo 'Java en 10 minutos' y autor 'Sergio Barros'. Los puntos 2 a 5 dan el mismo resultado con cualquiera de las dos versiones.

2. Imprimir todos los documentos de la colección 'libros'.

```
db.libros.find().pretty()
```

3. Imprimir todos los libros de autores de nacionalidad 'Argentina'.

```
// Para consultar un subcampo se usa la notación con punto y entre comillas  
db.libros.find({ 'autor.nacionalidad': 'Argentina' })
```

Devuelve El aleph y Martin Fierro.

4. Imprimir los libros de 'Borges'.

```
db.libros.find({ 'autor.nombre': 'Borges' })
```

Devuelve El aleph (_id 1).

5. Imprimir todos los libros de nacionalidad 'Española' que cuestan 50 o más.

```
db.libros.find({ 'autor.nacionalidad': 'Española', precio: { $gte: 50 } })
```

Devuelve Aprende PHP (precio 50). Java en 10 minutos es español pero cuesta 45.

1.10 Problemas propuestos 1.10 - Arreglos con documentos embebidos

1. Cargar los documentos de la colección 'series' (el enunciado dice 'libros', pero los datos son de series).

```
use base1  
db.series.drop()  
  
db.series.insertOne({  
  _id: 1,  
  titulo: 'The big bang theory',  
  productor: 'Chuck Lorre',  
  actores: ['Johnny Galecki', 'Jim Parsons', 'Kaley Cuoco', 'Kunal Nayyar',  
            'Simon Helberg', 'Mayim Bialik', 'Melissa Rauch'],  
  temporada1: [  
    { capitulo1: { titulo: 'Piloto', audiencia: 8300000 } },  
    { capitulo2: { titulo: 'La hipótesis del Gran Cerebro', audiencia: 8700000 } },  
    { capitulo3: { titulo: 'El Corolario de el Gato con Botas', audiencia: 9200000 } }  
  ],  
  temporada2: [  
    { capitulo1: { titulo: 'El paradigma del pescado malo', audiencia: 10000000 } },  
    { capitulo2: { titulo: 'La topología de la bragueta', audiencia: 11000000 } }  
  ]  
})
```

```
db.series.insertOne({
  _id: 2,
  titulo: 'The Walking Dead',
  productor: 'Robert Kirkman',
  actores: ['Andrew Lincoln', 'Jon Bernthal', 'Sarah Wayne Callies',
    'Laurie Holden', 'Jeffrey DeMunn', 'Steven Yeun'],
  temporada1: [
    { capitulo1: { titulo: 'TS 19', audiencia: 7000000 } },
    { capitulo2: { titulo: 'Wildfire', audiencia: 8200000 } },
    { capitulo3: { titulo: 'Díselo a las ranas', audiencia: 9100000 } }
  ],
  temporada2: [
    { capitulo1: { titulo: 'Lo que queda por delante', audiencia: 12000000 } },
    { capitulo2: { titulo: 'Sangría', audiencia: 13000000 } }
  ]
})
```

2. Imprimir todos los documentos de la colección 'series'.

```
db.series.find().pretty()
```

3. Imprimir los datos de todas las temporadas de la serie 'The big bang theory'.

```
db.series.find(
  { titulo: 'The big bang theory' },
  { _id: 0, titulo: 1, temporada1: 1, temporada2: 1 }
).pretty()
```

Muestra el título y los arreglos temporada1 y temporada2 completos.

4. Imprimir los datos de todos los capítulos de la primer temporada de la serie de 'The big bang theory'.

```
db.series.find(
  { titulo: 'The big bang theory' },
  { _id: 0, temporada1: 1 }
).pretty()
```

Muestra los 3 capítulos de la temporada 1: Piloto, La hipótesis del Gran Cerebro y El Corolario de el Gato con Botas.

5. Imprimir solo el primer capítulo de la primer temporada de 'The Walking Dead'.

```
// $slice: 1 recupera solo el primer elemento del arreglo temporada1
db.series.find(
  { titulo: 'The Walking Dead' },
  { _id: 0, titulo: 1, temporada1: { $slice: 1 } }
).pretty()
```

Resultado: el capítulo 1 de la temporada 1 ('TS 19', audiencia 7000000). El operador \$slice en la projection permite traer solo ese elemento del arreglo.

1.11

Problemas propuestos 1.11 - Tipo de dato Date

1. Crear la colección 'alumnos' en la base de datos 'base1' (eliminar la colección previamente) y cargar 3 documentos.

```
use base1
db.alumnos.drop()

db.alumnos.insertMany([
  { _id: 20456123, apellido: 'Gonzalez', nombre: 'Ana',
    domicilio: 'Colon 123', fechanacimiento: new Date(1990, 7, 15) },
  { _id: 45123845, apellido: 'Juarez', nombre: 'Bernardo',
    domicilio: 'Sucre 456', fechanacimiento: new Date(1964, 0, 1) },
  { _id: 16567512, apellido: 'Perez', nombre: 'Laura',
    domicilio: '21 de Septiembre 3233', fechanacimiento: new Date(1972, 3, 2) }
])
```

En JavaScript los meses van de 0 a 11, por eso `new Date(1990, 7, 15)` es el 15 de agosto de 1990.

2. Imprimir todos los documentos de la colección alumnos.

```
db.alumnos.find().pretty()
```

3. Imprimir solo el apellido y la fecha de nacimiento.

```
db.alumnos.find({}, { _id: 0, apellido: 1, fechanacimiento: 1 })
```

4. Imprimir todos los datos ordenados por la fecha de nacimiento de mayor a menor.

```
// -1 ordena de forma descendente: primero la fecha más reciente
db.alumnos.find().sort({ fechanacimiento: -1 }).pretty()
```

Orden de salida: Gonzalez (1990), Perez (1972) y Juarez (1964).

5. Imprimir todos los alumnos que nacieron a partir de 1970.

```
// Se compara contra el 1 de enero de 1970 (mes 0)
db.alumnos.find({ fechanacimiento: { $gte: new Date(1970, 0, 1) } }).pretty()
```

Devuelve Gonzalez Ana (1990) y Perez Laura (1972).